

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

**ACTIVIDAD LABORATORIO NO.2
PROGRAMACIÓN CON C BAJO UNIX**

PRESENTADO POR:
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING LAERTE ALBERTO FERREIRA CASTRO

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
02 DE JUNIO
2025

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Tabla de Contenido

RESUMEN	3
INTRODUCCIÓN.....	3
DESARROLLO ACTIVIDAD	4
Revisión de los temas.....	4
Tema 10: Compilación y aspectos básicos de C en Linux.....	4
Tema 11: Llamadas al sistema para manejo de archivos y directorios	5
Tema 12: Procesos, señales y comunicación en UNIX	6
Programa 1: Leer archivo con llamadas al sistema.....	7
Crear archivo fuente:.....	7
Ejecución del Programa 1.....	10
Creo archivo para lectura de nombre archivo.txt:	10
Guardar la salida.....	10
Programa 2: Crear proceso hijo y ejecutarlo	11
Crear archivo fuente:.....	11
Ejecución del Programa 2.....	14
Compilo el programa:	14
Guardar la salida.....	15
Programa 3: Comunicación entre procesos con pipe	16
Crear archivo fuente:.....	16
Ejecución del Programa 3.....	19
Compilo el programa:	19
Guardar la salida.....	19
CONCLUSIÓN DE LA ACTIVIDAD.....	20
BIBLIOGRAFÍA.....	20

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

RESUMEN

Este laboratorio me permitió fortalecer mis competencias en el uso de máquinas virtuales, administración de sistemas y configuración de entornos Linux, con un enfoque práctico en la programación en lenguaje C bajo UNIX y el uso de llamadas al sistema. Para ello, utilicé la herramienta VirtualBox junto con una imagen ISO de Linux Mint, lo que me permitió disponer de un entorno seguro y controlado para ejecutar y probar código sin afectar el sistema operativo principal.

Comencé descargando e instalando VirtualBox sobre mi sistema Windows. Luego, accedí a la página oficial de Linux Mint y descargué la ISO correspondiente, la cual monté como disco de arranque para crear una máquina virtual desde cero. Durante el proceso de instalación del sistema operativo, realicé la configuración inicial del entorno, incluyendo idioma, zona horaria, usuario y disco virtual.

Una vez instalado Linux Mint, me aseguré de contar con el compilador GCC y otras herramientas esenciales para el desarrollo en C. Esto me permitió compilar y ejecutar programas que hacen uso de llamadas al sistema como fork, exec, pipe, open, read, write, entre otras.

Durante el desarrollo de este laboratorio, creé varios programas en lenguaje C que interactúan directamente con el sistema operativo a través de dichas llamadas, simulando funcionalidades propias de la administración de procesos, comunicación entre procesos y gestión de archivos. La ejecución de estos programas en la terminal de Linux reforzó mis conocimientos tanto de programación como del funcionamiento interno de UNIX.

Asimismo, redirigí la salida de los programas a archivos de texto para documentar los resultados de las ejecuciones, facilitando la construcción de la memoria del laboratorio. Esta experiencia fue clave para comprender de manera práctica cómo los procesos se crean, comunican y terminan, además del manejo del sistema de archivos a bajo nivel.

En conclusión, este laboratorio no solo me permitió poner en práctica conceptos fundamentales de los sistemas operativos avanzados, sino también afianzar mis habilidades técnicas en un entorno realista gracias a la virtualización con VirtualBox. La integración de programación en C con herramientas del sistema Linux fue una experiencia enriquecedora que complementó perfectamente la teoría vista en los temas 10, 11 y 12.

INTRODUCCIÓN

En este laboratorio se busca fortalecer las competencias en la virtualización de sistemas operativos y, específicamente, en el desarrollo de programas en lenguaje C dentro de un entorno Linux, haciendo uso de llamadas al sistema propias de UNIX.

A través del uso de VirtualBox y la distribución Linux Mint, se implementó una máquina virtual completamente funcional que sirvió como plataforma segura para programar, compilar y ejecutar código en C. Este entorno aislado permite realizar pruebas sobre el

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

comportamiento del sistema operativo ante llamadas como fork, exec, pipe, open, read, write, entre otras, sin poner en riesgo la integridad del sistema anfitrión.

El objetivo principal de esta práctica es simular un entorno real de trabajo en Linux que permita comprender y aplicar de manera práctica los conceptos relacionados con los sistemas operativos avanzados, como la gestión de procesos, el uso del sistema de archivos a bajo nivel, la comunicación entre procesos (IPC) y el manejo directo de recursos mediante llamadas al sistema.

Gracias al uso de una imagen ISO de Linux Mint ejecutada en VirtualBox, se configuró el entorno necesario para llevar a cabo la compilación con gcc y ejecutar los distintos programas desarrollados durante el laboratorio. Esta práctica incluyó la asignación de hardware virtual (RAM, CPU y disco), la instalación del sistema operativo y la validación de herramientas esenciales para el desarrollo en C.

En resumen, este laboratorio fue fundamental para introducir un entorno Linux funcional y controlado, desde el cual se ejecutaron y analizaron programas que interactúan directamente con el núcleo del sistema operativo. La experiencia permitió consolidar conocimientos teóricos y sentar las bases necesarias para abordar tareas más avanzadas en la administración y programación de sistemas UNIX.

DESARROLLO ACTIVIDAD

El presente laboratorio tiene como objetivo poner en práctica los conocimientos adquiridos sobre la programación en lenguaje C bajo entornos UNIX/Linux, haciendo uso del compilador gcc y diversas llamadas al sistema. A través de la creación y ejecución de programas en C, se busca reforzar la comprensión del funcionamiento interno del sistema operativo, abordando temas como la gestión de procesos, comunicación entre procesos, y manipulación de archivos a bajo nivel.

Esta práctica permite consolidar habilidades esenciales para el desarrollo de software que interactúa directamente con el núcleo del sistema, utilizando funciones como fork, exec, pipe, open, read y write, entre otras. A continuación, procedo con la configuración del entorno y luego con una explicación detallada de los programas desarrollados y sus salidas dentro del sistema Linux.

Revisión de los temas

Para iniciar este laboratorio lo primero en mi caso fue dar una revisión de los temas de los cuales a continuación dejo un repaso de lo aprendido:

Tema 10: Compilación y aspectos básicos de C en Linux

Este tema se centra en los aspectos fundamentales de la compilación de programas en lenguaje C utilizando el compilador gcc en entornos UNIX/Linux. Algunos puntos clave incluyen o mejor los puntos clave que logre denotar son:

Compilación con gcc:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

- gcc archivo.c compila y genera un ejecutable por defecto llamado a.out.
- gcc -o programa archivo.c permite asignar un nombre personalizado al ejecutable.
- gcc -c archivo.c genera un archivo objeto .o sin enlazar.

Errores comunes con scanf:

- Declarar punteros sin reservar memoria: Ejemplo: char *cadena; scanf("%s", cadena); → Error en tiempo de ejecución.
- Uso incorrecto del formato: asignaciones a tipos incompatibles provocan errores en ejecución o comportamiento inesperado.

Variables y estructuras comunes:

- Uso correcto de arrays y punteros (char, int, etc.).
- Comprensión de la estructura de main(int argc, char *argv[]), donde:
 - argc: número de argumentos.
 - argv[]: arreglo de punteros a cadenas que representan los argumentos.
- Errores comunes de ejecución:
 - Acceso fuera de los límites de un array.
 - Mal uso de errno para capturar el código del último error.

Tema 11: Llamadas al sistema para manejo de archivos y directorios

Este tema abarca el uso de llamadas al sistema para interactuar directamente con el sistema de archivos. Algunas de las más relevantes o los puntos clave que logre denotar son:

Manejo de archivos:

- open(): abre o crea archivos.
- read(): lee datos desde un descriptor de archivo.
- write(): escribe datos.
- close(): cierra el descriptor.
- lseek(): cambia el puntero de lectura/escritura.

Información y permisos:

- stat(): obtiene información del archivo (tamaño, permisos, tipo).
- chmod(): cambia los permisos.
- chown(): cambia propietario.
- access(): comprueba permisos de acceso.

Manejo de directorios:

- mkdir(), rmdir(): crea y elimina directorios.
- opendir(), readdir(), closedir(): navegación en directorios.
- link() y unlink(): gestión de enlaces duros.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

- `symlink()`: creación de enlaces simbólicos.

Otros:

- `mount()`, `umount()`: montaje y desmontaje de sistemas de archivos.
- `sync()`: sincroniza operaciones pendientes de escritura en disco.

Tema 12: Procesos, señales y comunicación en UNIX

Este tema trata sobre la creación, ejecución, control y comunicación de procesos en sistemas UNIX/Linux, A continuación, los puntos clave que logre denotar:

Creación de procesos:

- `fork()`: crea un nuevo proceso hijo duplicado.
- `wait()`: espera a que un proceso hijo termine.
- `exit()`: termina el proceso actual.
- `getpid()`, `getppid()`: obtienen el PID del proceso y su padre.

Reemplazo del proceso:

- `exec()` y sus variantes (`execlp`, `execvp`, etc.): reemplazan el código del proceso actual por otro programa.

Manejo de señales:

- `signal()`, `kill()`: definen manejadores o envían señales a procesos.
- `pause()`, `alarm()`: espera o activa señales temporizadas.

Comunicación entre procesos (IPC):

- Tuberías anónimas (`pipe`): permiten la comunicación entre procesos emparentados (padre-hijo).
- Tuberías con nombre (`mknfifo`): permiten comunicación entre procesos independientes.
- `popen()` y `pclose()`: abren procesos como si fueran archivos, muy útiles para redirección de entrada/salida.

Importante:

- Al ejecutar `fork()`, el proceso hijo hereda una copia del espacio de memoria del padre.
- Al ejecutar `exec()`, el proceso actual se reemplaza completamente, comenzando desde `main()` del nuevo programa.

Ahora ya que he dado una explicación de los puntos clave de cada uno de los temas ahora si procedo a ejecutar un desarrollo para cada uno de las actividades del laboratorio, entonces para esto voy a ejecutar 3 programas completos para cubrir los conceptos clave:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Programa 1: Leer archivo con llamadas al sistema

Este programa en lenguaje C realiza la lectura del contenido de un archivo de texto utilizando llamadas al sistema UNIX (open, read, close) en lugar de funciones estándar como fopen o fread. A continuación, su respectivo análisis:

Crear archivo fuente:

Lo primero fue crear el archivo:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano leer_archivo.co

```

Ahora iniciamos con el siguiente código:

```

GNU nano 7.2 leer_archivo.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

```

Donde:

- <fcntl.h>: contiene la declaración de la función open() y las constantes como O_RDONLY.
- <unistd.h>: proporciona acceso a funciones del sistema como read() y close().
- <stdio.h>: se usa para printf() y perror().

Ahora ingresamos esta línea:

```

GNU nano 7.2 leer_archivo.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("archivo.txt", O_RDONLY);

```

Donde:

- open() abre el archivo archivo.txt en modo solo lectura (O_RDONLY).
- Si el archivo no existe o hay un error, fd será -1.

Ahora para revisar el error en caso ejecutamos estas líneas:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

GNU nano 7.2 leer_archivo.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("archivo.txt", O_RDONLY);
    if (fd < 0) {
        perror("Error al abrir el archivo");
        return 1;
    }
}

```

Donde:

Si hay un error, se muestra un mensaje descriptivo con `perror()` y se termina el programa con código 1.

Ahora declaramos el bufer y para esto ejecutamos:

```

GNU nano 7.2 leer_archivo.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("archivo.txt", O_RDONLY);
    if (fd < 0) {
        perror("Error al abrir el archivo");
        return 1;
    }

    char buffer[100];
    ssize_t bytes = read(fd, buffer, sizeof(buffer) - 1);
}

```

Donde:

- Se declara un buffer de 100 bytes.
- `read()` intenta leer hasta 99 bytes (dejando 1 byte para '\0' al final) desde el archivo hacia el buffer.
- `bytes` guarda el número de bytes realmente leídos.

Ahora se procede con la impresión:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

GNU nano 7.2 leer_archivo.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("archivo.txt", O_RDONLY);
    if (fd < 0) {
        perror("Error al abrir el archivo");
        return 1;
    }

    char buffer[100];
    ssize_t bytes = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes >= 0) {
        buffer[bytes] = '\0';
        printf("Contenido leído:\n%s\n", buffer);
    } else {
        perror("Error al leer");
    }
}

```

Donde:

- Si ocurre un error en read(), se muestra un mensaje.

Y por último procedo con el cierre:

```

GNU nano 7.2 leer_archivo.c
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    int fd = open("archivo.txt", O_RDONLY);
    if (fd < 0) {
        perror("Error al abrir el archivo");
        return 1;
    }

    char buffer[100];
    ssize_t bytes = read(fd, buffer, sizeof(buffer) - 1);
    if (bytes >= 0) {
        buffer[bytes] = '\0';
        printf("Contenido leído:\n%s\n", buffer);
    } else {
        perror("Error al leer");
    }

    close(fd);
    return 0;
}

```

^G Ayuda ^O Guardar ^W Buscar ^K Cortar ^T Ejecutar ^C Ubicación
 ^X Salir ^R Leer fich. ^\ Reemplazar ^U Pegar ^J Justificar ^_ Ir a línea

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Donde:

- Se cierra el archivo con `close(fd)` y el programa finaliza correctamente con `return 0`.

Ejecución del Programa 1

Ahora para ejecutar el programa entonces:

Creo archivo para lectura de nombre archivo.txt:

Esto es necesario ya que es el archivo de donde se va a ejecutar la lectura, en este caso lo denoto como "Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo Unix."

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano leer_archivo.c
alejo@alejo-VirtualBox:~$ echo "Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo Unix." > archivo.txt
alejo@alejo-VirtualBox:~$

```

Y luego que he creado con esta línea de código el archivo.txt, procedo a compilar con el siguiente código:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano leer_archivo.c
alejo@alejo-VirtualBox:~$ echo "Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo Unix." > archivo.txt
alejo@alejo-VirtualBox:~$ gcc -o leer leer_archivo.c
alejo@alejo-VirtualBox:~$

```

Y esto lo que hace es generar un ejecutable de nombre leer y por ende procedo a dar la ejecución con el siguiente código:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano leer_archivo.c
alejo@alejo-VirtualBox:~$ echo "Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo Unix." > archivo.txt
alejo@alejo-VirtualBox:~$ gcc -o leer leer_archivo.c
alejo@alejo-VirtualBox:~$ ./leer
Contenido leído:
Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo Unix.
alejo@alejo-VirtualBox:~$

```

Y como se puede denotar la respuesta es la correcta.

Guardar la salida

Ahora si necesito guardar la salida para incluirla en la consola entonces ejecuto el siguiente código:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano leer_archivo.c
alejo@alejo-VirtualBox:~$ echo "Holā, este es el desarrollo del primer programa del laboratorio No
alejo@alejo-VirtualBox:~$ gcc -o leer leer_archivo.c
alejo@alejo-VirtualBox:~$ ./leer
Contenido leído:
Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo U
alejo@alejo-VirtualBox:~$ ./leer > salida_programa1.txt
alejo@alejo-VirtualBox:~$

```

Y por ende ya puedo abrir esta salida siempre con:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano leer_archivo.c
alejo@alejo-VirtualBox:~$ echo "Holā, este es el desarrollo del primer programa del laboratorio No
alejo@alejo-VirtualBox:~$ gcc -o leer leer_archivo.c
alejo@alejo-VirtualBox:~$ ./leer
Contenido leído:
Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo U
alejo@alejo-VirtualBox:~$ ./leer > salida_programa1.txt
alejo@alejo-VirtualBox:~$ cat salida_programa1.txt
Contenido leído:
Hola, este es el desarrollo del primer programa del laboratorio No. 2 de programación con C bajo U
alejo@alejo-VirtualBox:~$

```

Y como podemos ver todo quedo correctamente.

Programa 2: Crear proceso hijo y ejecutarlo

Este programa demuestra el uso de procesos en UNIX/Linux, mediante las llamadas al sistema `fork()`, `execlp()` y `wait()`. Se muestra cómo un proceso padre crea un hijo, el cual reemplaza su ejecución con un comando del sistema. A continuación, su respectivo análisis:

Crear archivo fuente:

Lo primero entonces al igual que el anterior programa es crear el archivo:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano proceso_ls.c
alejo@alejo-VirtualBox:~$

```

Ahora iniciamos con el siguiente código:

```

GNU nano 7.2 proceso_ls.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

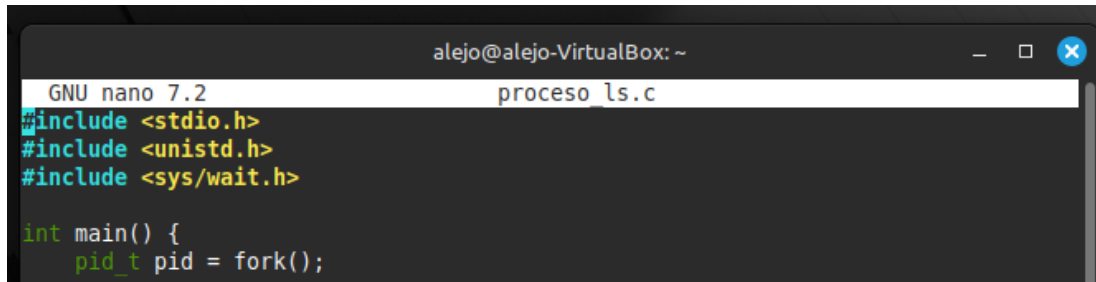
```

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Donde:

- `stdio.h`: Para imprimir mensajes (`printf`, `perror`).
- `unistd.h`: Proporciona acceso a `fork()`, `execlp()`, `getpid()`.
- `sys/wait.h`: Contiene `wait()`, que permite al padre esperar a que el hijo finalice.

Ahora ingresamos esta línea:



```

GNU nano 7.2 proceso ls.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

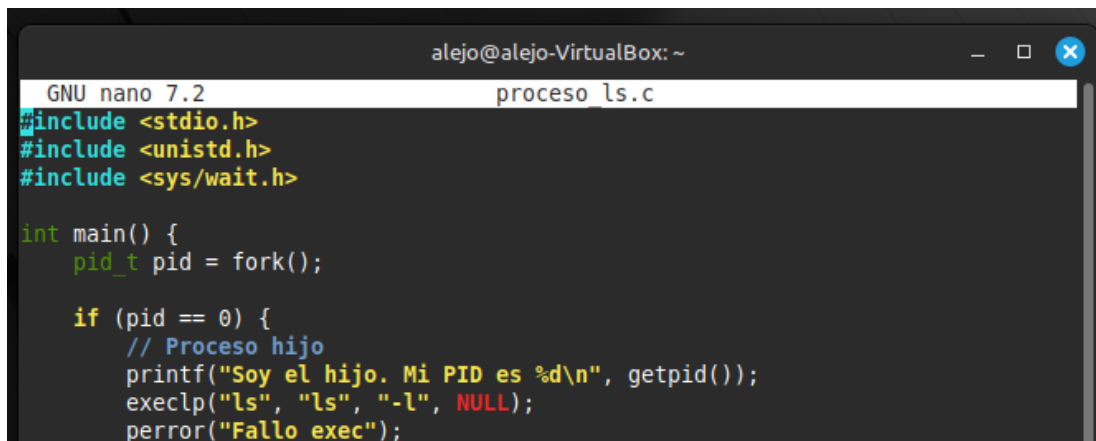
int main() {
    pid_t pid = fork();

```

Donde:

- `fork()` crea un nuevo proceso duplicado del proceso actual.
- Devuelve:
 - 0 = en el proceso hijo.
 - > 0 = en el proceso padre (valor es el PID del hijo).
 - < 0 = si hubo un error al crear el proceso.

Ahora se procede con la impresión:



```

GNU nano 7.2 proceso ls.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Proceso hijo
        printf("Soy el hijo. Mi PID es %d\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        perror("Fallo exec");
    }

```

Donde:

- Si estamos en el hijo (`pid == 0`):
 - Se imprime su PID.
 - Se ejecuta el comando `ls -l` usando `execlp()` (que busca el ejecutable en el PATH).
 - Si `execlp()` falla, se muestra un error con `perror()`.

Es importante precisar que si `execlp()` funciona correctamente, el resto del código del hijo no se ejecuta, ya que es reemplazado por el nuevo programa.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Ahora ejecutamos el proceso de espera del padre con:

```

GNU nano 7.2 proceso_ls.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Proceso hijo
        printf("Soy el hijo. Mi PID es %d\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        perror("Fallo exec");
    } else if (pid > 0) {
        // Proceso padre
        wait(NULL);
        printf("Proceso hijo finalizado\n");
    }
}

```

Donde:

- El padre espera con `wait(NULL)` a que el hijo termine.
- Luego imprime que el hijo ha finalizado.

Y por último en dado caso de que haya una falla en `fork()`, se muestra el mensaje de error con el siguiente código:

```

GNU nano 7.2 proceso_ls.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Proceso hijo
        printf("Soy el hijo. Mi PID es %d\n", getpid());
        execlp("ls", "ls", "-l", NULL);
        perror("Fallo exec");
    } else if (pid > 0) {
        // Proceso padre
        wait(NULL);
        printf("Proceso hijo finalizado\n");
    } else {
        perror("Error al crear proceso");
    }
}

```

[22 líneas leídas]

[^]G Ayuda [^]O Guardar [^]W Buscar [^]K Cortar [^]T Ejecutar [^]C Ubicación
[^]X Salir [^]R Leer fich. [^]\ Reemplazar [^]U Pegar [^]J Justificar [^]/ Ir a línea

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Ejecución del Programa 2

Ahora para ejecutar el programa entonces:

Compilo el programa:

Procedo a compilar con el siguiente código:

```
alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano proceso_ls.c
alejo@alejo-VirtualBox:~$ gcc -o proceso_ls proceso_ls.c
```

Y esto lo que hace es generar un ejecutable de nombre proceso_ls y por ende procedo a dar la ejecución con el siguiente código:

```
alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano proceso_ls.c
alejo@alejo-VirtualBox:~$ gcc -o proceso_ls proceso_ls.c
alejo@alejo-VirtualBox:~$ ./proceso_ls
Soy el hijo. Mi PID es 4940
total 136
-rw-rw-r-- 1 alejo alejo 104 jun  2 22:34 archivo.txt
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Descargas
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Documentos
drwxrwxr-x 2 alejo alejo 4096 may 27 00:34 Ejercicios
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Escritorio
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Imágenes
-rwxrwxr-x 1 alejo alejo 16184 jun  2 22:35 leer
-rw-rw-r-- 1 alejo alejo 471 jun  2 22:19 leer_archivo.c
-rw-rw-r-- 1 alejo alejo 471 jun  2 22:15 leer_archivo.co
-rwxrwxr-x 1 alejo alejo 758 may 26 08:20 modificar_path.sh
-rwxrwxr-x 1 alejo alejo 719 may 26 08:32 modificar_script_para_llamar_usuario
conectado.sh
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Música
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Plantillas
-rwxrwxr-x 1 alejo alejo 16216 jun  2 22:57 proceso_ls
-rw-rw-r-- 1 alejo alejo 465 jun  2 22:48 proceso_ls.c
-rwxrwxr-x 1 alejo alejo 15960 jun  2 20:51 programa
-rwxrwxr-x 1 alejo alejo 15968 jun  2 21:31 programa2
-rw-rw-r-- 1 alejo alejo 234 jun  2 21:26 programa2.c
-rw-rw-r-- 1 alejo alejo 115 jun  2 20:55 programa.c
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Público
-rw-rw-r-- 1 alejo alejo 118 jun  2 22:39 salida_programa1.txt
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Vídeos
Proceso hijo finalizado
alejo@alejo-VirtualBox:~$
```

Y como se puede denotar la respuesta es la correcta:

- Soy el hijo. Mi PID es 4567
- Total, de archivos y permisos listados 136
- Proceso hijo finalizado

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Guardar la salida

Ahora si necesito guardar la salida para incluirla en la consola entonces ejecuto el siguiente código:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano proceso_ls.c
alejo@alejo-VirtualBox:~$ gcc -o proceso_ls proceso_ls.c
alejo@alejo-VirtualBox:~$ ./proceso_ls
Soy el hijo. Mi PID es 4940
total 136
-rw-rw-r-- 1 alejo alejo 104 jun 2 22:34 archivo.txt
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Descargas
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Documentos
drwxrwxr-x 2 alejo alejo 4096 may 27 00:34 Ejercicios
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Escritorio
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Imágenes
-rwxrwxr-x 1 alejo alejo 16184 jun 2 22:35 leer
-rw-rw-r-- 1 alejo alejo 471 jun 2 22:19 leer_archivo.c
-rw-rw-r-- 1 alejo alejo 471 jun 2 22:15 leer_archivo.co
-rwxrwxr-x 1 alejo alejo 758 may 26 08:20 modificar_path.sh
-rwxrwxr-x 1 alejo alejo 719 may 26 08:32 modificar_script_para_llamar_usuario
conectado.sh
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Música
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Plantillas
-rwxrwxr-x 1 alejo alejo 16216 jun 2 22:57 proceso_ls
-rw-rw-r-- 1 alejo alejo 465 jun 2 22:48 proceso_ls.c
-rwxrwxr-x 1 alejo alejo 15960 jun 2 20:51 programa
-rwxrwxr-x 1 alejo alejo 15968 jun 2 21:31 programa2
-rw-rw-r-- 1 alejo alejo 234 jun 2 21:26 programa2.c
-rw-rw-r-- 1 alejo alejo 115 jun 2 20:55 programa.c
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Público
-rw-rw-r-- 1 alejo alejo 118 jun 2 22:39 salida_programa1.txt
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Vídeos
Proceso hijo finalizado
alejo@alejo-VirtualBox:~$ ./proceso_ls > salida_programa2.txt

```

Y por ende ya puedo abrir esta salida siempre con:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

alejo@alejo-VirtualBox: ~
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Público
-rw-rw-r-- 1 alejo alejo 118 jun 2 22:39 salida_programa1.txt
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Videos
Proceso hijo finalizado
alejo@alejo-VirtualBox:~$ ./proceso_ls > salida_programa2.txt
alejo@alejo-VirtualBox:~$ cat salida_programa2.txt
total 136
-rw-rw-r-- 1 alejo alejo 104 jun 2 22:34 archivo.txt
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Descargas
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Documentos
drwxrwxr-x 2 alejo alejo 4096 may 27 00:34 Ejercicios
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Escritorio
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Imágenes
-rwxrwxr-x 1 alejo alejo 16184 jun 2 22:35 leer
-rw-rw-r-- 1 alejo alejo 471 jun 2 22:19 leer_archivo.c
-rw-rw-r-- 1 alejo alejo 471 jun 2 22:15 leer_archivo.co
-rwxrwxr-x 1 alejo alejo 758 may 26 08:20 modificar_path.sh
-rwxrwxr-x 1 alejo alejo 719 may 26 08:32 modificar_script_para_llamar_usuario
conectado.sh
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Música
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Plantillas
-rwxrwxr-x 1 alejo alejo 16216 jun 2 22:57 proceso_ls
-rw-rw-r-- 1 alejo alejo 465 jun 2 22:48 proceso_ls.c
-rwxrwxr-x 1 alejo alejo 15960 jun 2 20:51 programa
-rwxrwxr-x 1 alejo alejo 15968 jun 2 21:31 programa2
-rw-rw-r-- 1 alejo alejo 234 jun 2 21:26 programa2.c
-rw-rw-r-- 1 alejo alejo 115 jun 2 20:55 programa.c
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Público
-rw-rw-r-- 1 alejo alejo 118 jun 2 22:39 salida_programa1.txt
-rw-rw-r-- 1 alejo alejo 0 jun 2 23:00 salida_programa2.txt
drwxr-xr-x 2 alejo alejo 4096 may 26 07:03 Videos
Proceso hijo finalizado
alejo@alejo-VirtualBox:~$

```

Y como podemos ver todo quedo correctamente.

Programa 3: Comunicación entre procesos con pipe

Este programa muestra cómo dos procesos “padre e hijo” se comunican entre sí utilizando una tubería (pipe). La comunicación es unidireccional: el padre escribe un mensaje, y el hijo lo lee. A continuación, su respectivo análisis:

Crear archivo fuente:

Lo primero entonces al igual que el anterior programa es crear el archivo:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano pipe_proceso.c
alejo@alejo-VirtualBox:~$

```

Ahora iniciamos con el siguiente código:

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

```

alejo@alejo-VirtualBox: ~
GNU nano 7.2 pipe_proceso.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

```

Donde:

- stdio.h: funciones estándar (printf).
- unistd.h: llamadas al sistema como pipe(), fork(), read(), write().
- string.h: para usar strlen().

Ahora ingresamos esta línea:

```

alejo@alejo-VirtualBox: ~
GNU nano 7.2 pipe_proceso.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pipe(fd);
}

```

Donde:

- Se declara un arreglo de 2 enteros: fd[0] será el extremo de lectura, y fd[1] el extremo de escritura.
- pipe(fd) crea una tubería anónima. Ahora fd contiene descriptores de archivo conectados.

Ahora se procede con:

```

alejo@alejo-VirtualBox: ~
GNU nano 7.2 pipe_proceso.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pipe(fd);

    if (fork() == 0) {
        // Hijo
        close(fd[1]); // Cierra escritura
    }
}

```

Donde:

- El proceso hijo cierra el extremo de escritura para que solo lea datos.
- Es importante cerrar el extremo no usado para evitar bloqueos.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Ahora ejecutamos la lectura del hijo con:



```

GNU nano 7.2 pipe_proceso.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pipe(fd);

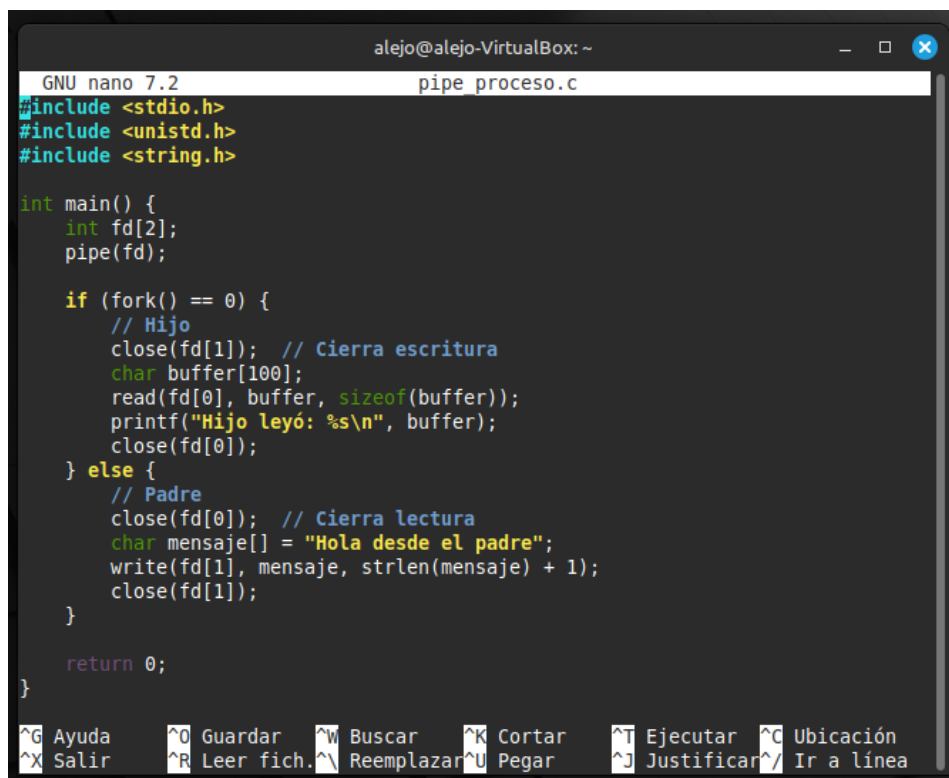
    if (fork() == 0) {
        // Hijo
        close(fd[1]); // Cierra escritura
        char buffer[100];
        read(fd[0], buffer, sizeof(buffer));
        printf("Hijo leyó: %s\n", buffer);
        close(fd[0]);
    }
}

```

Donde:

- El hijo lee desde la tubería en fd[0] y lo imprime.
- Finalmente cierra el extremo de lectura.

Y por último el padre cierra el extremo de lectura, escribe el mensaje en la tubería y luego cierra el extremo de escritura con el siguiente código:



```

GNU nano 7.2 pipe_proceso.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pipe(fd);

    if (fork() == 0) {
        // Hijo
        close(fd[1]); // Cierra escritura
        char buffer[100];
        read(fd[0], buffer, sizeof(buffer));
        printf("Hijo leyó: %s\n", buffer);
        close(fd[0]);
    } else {
        // Padre
        close(fd[0]); // Cierra lectura
        char mensaje[] = "Hola desde el padre";
        write(fd[1], mensaje, strlen(mensaje) + 1);
        close(fd[1]);
    }

    return 0;
}

```

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Ejecución del Programa 3

Ahora para ejecutar el programa entonces:

Compilo el programa:

Procedo a compilar con el siguiente código:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano pipe_proceso.c
alejo@alejo-VirtualBox:~$ gcc -o pipe_proceso pipe_proceso.c
alejo@alejo-VirtualBox:~$

```

Y esto lo que hace es generar un ejecutable de nombre pipe_proceso y por ende procedo a dar la ejecución con el siguiente código:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano pipe_proceso.c
alejo@alejo-VirtualBox:~$ gcc -o pipe_proceso pipe_proceso.c
alejo@alejo-VirtualBox:~$ ./pipe_proceso
Hijo leyó: Hola desde el padre
alejo@alejo-VirtualBox:~$

```

Y como se puede denotar la respuesta es la correcta:

- Hijo leyó: Hola desde el padre

Guardar la salida

Ahora si necesito guardar la salida para incluirla en la consola entonces ejecuto el siguiente código:

```

alejo@alejo-VirtualBox: ~
alejo@alejo-VirtualBox:~$ nano pipe_proceso.c
alejo@alejo-VirtualBox:~$ gcc -o pipe_proceso pipe_proceso.c
alejo@alejo-VirtualBox:~$ ./pipe_proceso
Hijo leyó: Hola desde el padre
alejo@alejo-VirtualBox:~$ ./pipe_proceso > salida_programa3.txt
cat salida_programa3.txt
Hijo leyó: Hola desde el padre
alejo@alejo-VirtualBox:~$

```

Por ende, ya puedo abrir esta salida siempre con “cat salida_programa3.txt”, y por ende como podemos ver todo quedo correctamente.

Asignatura	Datos del alumno	Fecha
Sistemas Operativos Avanzados	Apellidos: De Mendoza Tovar	02-06-2025
	Nombre: Alejandro	

Y con este último resumen explicativo de todos los programas finalizo el desarrollo de este laboratorio. **¡Mil gracias, respetado profesor por estos valiosos conocimientos aportados, de muy valiosa utilidad en mi ámbito profesional!**

CONCLUSIÓN DE LA ACTIVIDAD

Para concluir este trabajo, puedo afirmar que el uso de herramientas como VirtualBox y Linux Mint ha sido clave para comprender e implementar conceptos fundamentales de los sistemas operativos avanzados, particularmente en lo relacionado con la programación en lenguaje C y el manejo de llamadas al sistema bajo entornos UNIX/Linux. La virtualización no solo ofrece un entorno seguro para ejecutar código a bajo nivel, sino que también permite experimentar con confianza, aprender de los errores y profundizar en la interacción directa con el sistema operativo.

Durante el desarrollo del laboratorio, no solo se trató de escribir código en C, sino de entender cómo este código interactúa con el sistema operativo mediante funciones como `fork()`, `exec()`, `pipe()`, `open()`, `read()` y `write()`. Estas prácticas permitieron observar de forma práctica el comportamiento de los procesos, la gestión del sistema de archivos y los mecanismos de comunicación entre procesos.

Resalto especialmente cómo esta experiencia se convierte en una base sólida para abordar temas más avanzados como la administración de procesos, la sincronización, el control de recursos y la eficiencia en la ejecución de programas en tiempo real. Este tipo de prácticas permiten consolidar el conocimiento teórico desde una perspectiva práctica, crítica y aplicada.

Finalmente, esta experiencia no solo reforzó mis conocimientos técnicos, sino que también me permitió valorar la importancia de la virtualización como una herramienta pedagógica poderosa y replicable. Trabajar directamente con el sistema operativo, desde su instalación hasta la ejecución de programas de bajo nivel, representa un paso clave en mi formación como ingeniero informático. Muchas gracias, respetado y estimado profesor.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada en la búsqueda de información:

- Tema 10. Programación del sistema en Linux. Tema 11. Llamadas al sistema sobre ficheros. Tema 12. Llamadas al sistema sobre procesos.
- Clases virtuales con el profesor Ing, Laerte Alberto Ferreira Castro.
- Shotts, W. E. (2019). The Linux Command Line: A Complete Introduction (2nd ed.). No Starch Press. Manual práctico sobre Bash, comandos, scripting y automatización en Linux.
- Sobell, M. G. (2018). A Practical Guide to Linux Commands, Editors, and Shell Programming (4th ed.). Pearson Education. Detalles técnicos y ejemplos sobre nano, permisos (`chmod`), y programación de scripts.
- The Linux Documentation Project. (2024). Advanced Bash-Scripting Guide. Recuperado de: <https://tldp.org/LDP/abs/html/>. Guía avanzada para escribir y modificar scripts Bash.